

Reviewer Pack — Moment Matrix Sparsity Lower Bound for Max-CUT SOS Approxima...

Entry f58c235db097 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 16:25:47 UTC

Moment Matrix Sparsity Lower Bound for Max-CUT SOS Approximation

Entry ID: f58c235db097 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 16:25:47 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): SOS Refutation Degree for Max-CUT

Statement:

For any Max-CUT instance with n variables, any degree- d pseudoexpectation satisfying the SOS feasibility conditions must have a moment matrix with $\geq \Omega(n^2)$ non-zero entries. The sparsity threshold is $\Theta(n^2/d^2)$ for $d \geq 2$.

Rationale (proposer's reasoning):

The moment matrix encodes polynomial constraints of the SOS hierarchy. High sparsity would imply algebraic dependencies that could be exploited for lower-degree refutations, but the conjecture posits that strong approximation guarantees (like 0.879-approximation) require dense moment matrices, exposing a trade-off between sparsity and approximation quality.

Taxonomy category: SOS_DEGREE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 17653673121f5fcf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gram_schmidt(monomials):
    n = len(monomials)
    Q = []
    for i in range(n):
        q_i = monomials[i]
        for j in range(i):
            q_j = Q[j]
            proj = sum(q_i[k] * q_j[k] for k in range(len(q_i))) / sum(q_j[k]**2 for k in range(len(q_j)))
            q_i = [q_i[k] - proj * q_j[k] for k in range(len(q_i))]
        norm = math.sqrt(sum(q_i[k]**2 for k in range(len(q_i))))
        Q.append([q_i[k] / norm for k in range(len(q_i))])
    return Q

def moment_matrix(monomials, d):
    n = len(monomials)
    M = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(n):
        for j in range(n):
            if abs(monomials[i][j]) > 1e-9:
                M[i][j] = monomials[i][j]
                M[j][i] = monomials[j][i]
    return M

def count_non_zero_entries(matrix):
    n = len(matrix)
    count = 0
    for i in range(n):
        for j in range(i, n):
            if matrix[i][j] != 0:
                count += 1
    return count

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
```

```

d = random.randint(2, 5)

# Generate a random Max-CUT instance
variables = [f'x{i}' for i in range(n)]
monomials = []
for i in range(n):
    for j in range(i + 1, n):
        monomials.append([0] * (n + 1))
        monomials[-1][i] = random.choice([-1, 1])
        monomials[-1][j] = random.choice([-1, 1])

# Construct the degree-d moment matrix
Q = gram_schmidt(monomials)
M = moment_matrix(Q, d)

# Count non-zero entries in the moment matrix
non_zero_count = count_non_zero_entries(M)

# Calculate the sparsity threshold
sparsity_threshold = n**2 / (d**2 * 4) #  $\Theta(n^2/d^2)$  is an overestimate for simplicity

# Check if the conjecture holds
conjecture_holds = non_zero_count >= sparsity_threshold

return {
    "metric_name": "non_zero_entries",
    "metric_value": non_zero_count,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"n={n}, d={d}, non_zero_count={non_zero_count}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    # Compute mean/std of metric_value and fraction of seeds where conjecture_holds
    total_non_zero_entries = sum(r["metric_value"] for r in results)
    total_instances_tested = sum(r["instances_tested"] for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    mean_non_zero_entries = total_non_zero_entries / total_instances_tested
    std_non_zero_entries = math.sqrt(sum((r["metric_value"] - mean_non_zero_entries)**2 for r in results) / (len(results) - 1))

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_non_zero_entries} std={std_non_zero_entries} support={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]))
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    else:

```

```
print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6ad841dd.py", line 91, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6ad841dd.py", line 65, in run_trial
    Q = gram_schmidt(monomials)
      ^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6ad841dd.py", line 28, in gram_schmidt
    Q.append([q_i[k] / norm for k in range(len(q_i))])
              ^~~~~~
```

ZeroDivisionError: float division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with division by zero error, preventing data collection. Cannot assess conjecture validity without successful runs. | next: Debug gram_schmidt implementation to avoid division by zero, then re-run experiments with multiple seeds

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	107643
2	propose	ollama_remote	qwen3:8b	0	0	111997
3	propose	ollama_remote	qwen3:8b	0	0	90976
4	preregistration	ollama_remote	qwen3:8b	0	0	24046
5	novelty	ollama_remote	qwen3:8b	0	0	20919
6	novelty	ollama_remote	qwen3:8b	0	0	14657
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14296
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11369
9	judge	ollama_remote	qwen3:8b	0	0	21837

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 417741 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/f58c235db097.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f58c235db097.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f58c235db097.tar.gz` (if generated)